Karadeniz Teknik Üniversitesi

Fen Fakültesi – Bilgisayar Bilimleri Bölümü

OBYS Tam Proje – Derlenebilir CMake Projesi

Bölüm 3 – Hafta 3.5

Yazılım Geliştirme II (BILB2000)

Bahar Dönemi – 3.5. Hafta

Bu Haftanın Planı

1. Proje İskelesi ve CMake
2. Varlık Struct'ları
3. Depo<K,V> ve Hata Yönetimi
4. Servis Katmanı
5. Dosyalama Katmanı
6. Derleme, Test ve Karşılaştırma
7. Özet

Proje İskelesi ve CMake

Neden Ayrı Bir Proje İskelesi?

Sorun

Şimdiye kadar her kavramı tek bir `.cpp` dosyasında denedik. Gerçek bir OBYS projesinde **onlarca** dosya, **birden fazla** hedef (program + test) ve **modüler başlık dosyaları** olacak.

Çözüm: CMake + Modüler Yapı

- Her katman kendi klasöründe
- Başlık dosyaları (`include/`) ile bağımlılık yönetimi
- CMake ile tek komutla derleme
- Ana program + test programı ayrı hedefler

Dizin Yapısı

CMake Proje Dizin Yapısı

obys_modern/	*Proje kök dizini*
`CMakeLists.txt`	*Derleme yapılandırması*
include/	*Başlık dosyaları*
varliklar/	*Veri yapıları (struct)*
`ogrenci.h`	*flat struct + enum class*
`ders.h`	
`ders_kaydi.h`	
`not_turleri.h`	
hatalar/	*Özel istisna sınıfları*
`obys_hatasi.h`	
`depo.h`	*template Depo<K,V> sınıfı*
`dosyalama.h`	*Binary dosyalama modülü*
servisler/	*İş mantığı katmanı*
`obys_servis.h`	
src/	*Kaynak dosyalar*
`main.cpp`	*Konsol arayüzü*
`servisler/obys_servis.cpp`	*Servis implementasyonu*
`test_main.cpp`	*Birim test dosyası*

CMakeLists.txt

```
cmake_minimum_required(VERSION 3.20)
project(Uygulama3 LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

include_directories(include)

# Ana calistirici
add_executable(Uygulama3
    src/main.cpp
    src/servisler/obys_servis.cpp
)

# Test calistirici
add_executable(TestUygulama3
    src/test_main.cpp
    src/servisler/obys_servis.cpp
)
```

Anahtar Noktalar

C++20 zorunlu · `include_directories` ile başlık yolu · İki ayrı hedef: `Uygulama3` + `TestUygulama3`

Varlık Struct'ları

Eski vs Yeni Tasarım

Eski: class + getter/setter

Gereksiz kapsülleme · Kalıtım zorunlu · Kod tekrarı

Yeni: flat struct

Doğrudan üye erişimi · Varsayılan başlangıç · Toplu başlatma (aggregate init)

Ogrenci ve Ders Struct'ları

ogrenci.h

```cpp
#pragma once
#include <string>

struct Ogrenci {
    int numara{};
    std::string isim;
    std::string soyisim;
    std::string dogum_tarihi;
    std::string cinsiyet;
};
```

ders.h

```cpp
#pragma once
#include <string>

struct Ders {
    std::string ders_kodu;
    std::string ders_adi;
};
```

Tasarım Kararları

- `#pragma once` – tekrarlı dahil etmeyi önler
- Varsayılan başlangıç: `int numara{}` → sıfırla
- Getter/setter yok → doğrudan `ogr.isim` erişimi

NotTuru ve DersKaydi

not_turleri.h

```cpp
#pragma once
#include <variant>
#include <string>

struct Muaf {
    std::string sebep{"Muaf"};
};

using NotTuru =
    std::variant<int,
                 std::string,
                 Muaf>;
```

ders_kaydi.h

```cpp
struct DersKaydi {
    int ogrenci_no{};
    std::string ders_kodu;
    NotTuru vize;
    NotTuru final_notu;
    NotTuru odev;

    // Ortalama: %30+%40+%30
    float ortalama() const;
};
```

variant ile Esneklik

Bir not; sayı (85), harf (`"AA"`) veya `Muaf{}` olabilir. Tür-güvenli ve genişletilebilir!

Depo<K,V> ve Hata Yönetimi

Depo<K,V> Template Sınıfı

```
template <typename K, typename V>
class Depo {
public:
    void ekle(const K& anahtar, const V& deger) {
        if (_elemanlar.contains(anahtar))
            throw std::runtime_error("Eleman zaten mevcut!");
        _elemanlar.emplace(anahtar, deger);
    }

    std::optional<V> bul(const K& anahtar) const {
        auto it = _elemanlar.find(anahtar);
        if (it != _elemanlar.end()) return it->second;
        return std::nullopt;
    }

    const V& getir(const K& anahtar) const {
        auto it = _elemanlar.find(anahtar);
        if (it == _elemanlar.end())
            throw std::runtime_error("Eleman bulunamadi!");
        return it->second;
    }

    void temizle() { _elemanlar.clear(); }
private:
    std::map<K, V> _elemanlar;
};
```

bul() vs getir() Stratejisi

`bul()` → optional

- Bulamamak **normal** bir durum
- `nullopt` döner, istisna yok
- Çağıran karar verir
- Kullanım: arama, kontrol

`getir()` → exception

- Eleman **olmalı**, yoksa hata
- İstisna fırlatır
- Referans döner (kopya yok)
- Kullanım: kesin beklenti

Kural

"Belki yok" → `bul()` (`optional`) "Olmalı!" → `getir()` (istisna)

OBYSHatasi Hiyerarşisi

```
class OBYSHatasi
    : public std::runtime_error {
public:
    using runtime_error::
          runtime_error;
};

class OgrenciHatasi
    : public OBYSHatasi {
public:
    using OBYSHatasi::OBYSHatasi;
};

class OgrenciBulunamadiHatasi
    : public OgrenciHatasi {
public:
    using OgrenciHatasi::
          OgrenciHatasi;
};
```

- std::runtime_error
  - OBYSHatasi
    - OgrenciHatasi
      - Bulunamadi
      - ZatenVar
    - DersHatasi
      - Bulunamadi
      - ZatenVar
    - KayitHatasi

using ile Kısa Tanım

using bildirimi ile üst sınıfın constructor'ını miras al → tekrar yazma!

Servis Katmanı

OBYSServis: Üç Depoyu Koordine Eder

```
class OBYSServis {
public:
    OBYSServis(Depo<int, Ogrenci>& ogrenciDepo,
               Depo<std::string, Ders>& dersDepo,
               Depo<int, DersKaydi>& dersKaydiDepo);

    void ogrenci_ekle(int numara, std::string_view isim,
                      std::string_view soyisim,
                      std::string_view dogum_tarihi,
                      std::string_view cinsiyet);

    void ders_ekle(std::string_view ders_kodu,
                   std::string_view ders_adi);

    void not_ekle(int ogrenci_no, std::string_view ders_kodu,
                  int vize, int final_notu, int odev);

    // Sorgular
    float sinif_ortalamasi(std::string_view ders_kodu) const;
    std::string ogrenci_transkript(int ogrenci_no) const;

private:
    Depo<int, Ogrenci>& _ogrenciDepo;
    Depo<std::string, Ders>& _dersDepo;
    Depo<int, DersKaydi>& _dersKaydiDepo;
};
```

Ranges Pipeline: Sınıf Ortalaması

```
float OBYSServis::sinif_ortalamasi(
    std::string_view ders_kodu) const
{
    std::string kod(ders_kodu);
    if (!_dersDepo.iceriyor_mu(kod))
        throw DersBulunamadiHatasi(
            std::format("{} kodlu ders bulunamadi!", kod));

    const auto& kayitlar = _dersKaydiDepo.tumunu_al();

    // ranges::views ile filtreleme
    auto filtreli = kayitlar
        | std::views::values
        | std::views::filter([&kod](const DersKaydi& k) {
              return k.ders_kodu == kod;
          });

    float toplam = 0.0f;
    int sayac = 0;
    for (const auto& kayit : filtreli) {
        toplam += kayit.ortalama();
        ++sayac;
    }
    return toplam / static_cast<float>(sayac);
}
```

string_view + format ile Transkript

```
std::string OBYSServis::ogrenci_transkript(int ogrenci_no) const {
    auto ogr_opt = _ogrenciDepo.bul(ogrenci_no);
    if (!ogr_opt.has_value())
        throw OgrenciBulunamadiHatasi(
            std::format("{} numarali ogrenci bulunamadi!", ogrenci_no));

    const auto& ogr = ogr_opt.value();
    std::string sonuc = std::format(
        "=== Transkript ===\nOgrenci: {} {} ({})\n",
        ogr.isim, ogr.soyisim, ogr.numara);

    const auto& kayitlar = _dersKaydiDepo.tumunu_al();
    auto ogr_kayitlari = kayitlar
        | std::views::values
        | std::views::filter([ogrenci_no](const DersKaydi& k) {
              return k.ogrenci_no == ogrenci_no;
          });

    for (const auto& kayit : ogr_kayitlari) {
        auto ders_opt = _dersDepo.bul(kayit.ders_kodu);
        std::string ders_adi = ders_opt.has_value()
            ? ders_opt.value().ders_adi : kayit.ders_kodu;
        sonuc += std::format("{:<8} {:<30} Ort: {:.1f}\n",
            kayit.ders_kodu, ders_adi, kayit.ortalama());
    }
    return sonuc;
}
```

Dosyalama Katmanı

reinterpret_cast ile POD Yazma/Okuma

Temel Fikir

POD (Plain Old Data) türleri bellekte ardışık baytlar olarak tutulur. Bu baytları doğrudan dosyaya yazıp geri okuyabiliriz.

```
// Genel sablonlar
template <typename T>
void dosyaya_yaz(std::ofstream& dosya, const T& veri) {
    dosya.write(
        reinterpret_cast<const char*>(&veri), sizeof(T));
}

template <typename T>
void dosyadan_oku(std::ifstream& dosya, T& veri) {
    dosya.read(
        reinterpret_cast<char*>(&veri), sizeof(T));
}
```

Dikkat

`reinterpret_cast` yalnızca POD türler (`int`, `float`, `char`) için güvenlidir. `std::string` POD **değildir**!

string: Boyut + Veri Kalıbı

```
// string ozel surumu: once boyut, sonra veri
template <>
void dosyaya_yaz<std::string>(std::ofstream& dosya,
                              const std::string& veri) {
    std::size_t boyut = veri.size();
    dosya.write(reinterpret_cast<const char*>(&boyut),
                sizeof(boyut));
    dosya.write(veri.data(),
                static_cast<std::streamsize>(boyut));
}

template <>
void dosyadan_oku<std::string>(std::ifstream& dosya,
                               std::string& veri) {
    std::size_t boyut = 0;
    dosya.read(reinterpret_cast<char*>(&boyut),
               sizeof(boyut));
    veri.resize(boyut);
    dosya.read(veri.data(),
               static_cast<std::streamsize>(boyut));
}
```

variant Serileştirme: index() + switch

```
// Yazma: once index(), sonra uygun turu yaz
void dosyaya_yaz(std::ofstream& dosya, const NotTuru& veri) {
    uint8_t idx = static_cast<uint8_t>(veri.index());
    dosya.write(reinterpret_cast<const char*>(&idx), 1);
    std::visit([&](const auto& val) {
        dosyaya_yaz(dosya, val);
    }, veri);
}

// Okuma: once index oku, sonra switch ile dogru turu oku
void dosyadan_oku(std::ifstream& dosya, NotTuru& veri) {
    uint8_t idx = 0;
    dosya.read(reinterpret_cast<char*>(&idx), 1);
    switch (idx) {
        case 0: { int v{}; dosyadan_oku(dosya, v);
                  veri = v; break; }
        case 1: { std::string v; dosyadan_oku(dosya, v);
                  veri = v; break; }
        case 2: { Muaf v; dosyadan_oku(dosya, v);
                  veri = v; break; }
    }
}
```

map ve Depo Serileştirme

```
// map<K,V>: boyut + key/value ciftleri
template <typename K, typename V>
void dosyaya_yaz(std::ofstream& dos, const std::map<K,V>& m) {
    std::size_t boyut = m.size();
    dos.write(reinterpret_cast<const char*>(&boyut),
              sizeof(boyut));
    for (const auto& [anahtar, deger] : m) {
        dosyaya_yaz(dos, anahtar);
        dosyaya_yaz(dos, deger);
    }
}

// Depo<K,V>: tumunu_al() ile map'i yaz
template <typename K, typename V>
void dosyaya_yaz(std::ofstream& dos, const Depo<K,V>& depo) {
    dosyaya_yaz(dos, depo.tumunu_al());
}
```

Katmanlı Tasarım

Her tür kendi `dosyaya_yaz` / `dosyadan_oku` çiftine sahip. Üst katman alt katmanı çağırır → **kompozisyon**.

Ana Programda Dosyalama Entegrasyonu

```
// Dosyadan yukle (varsa)
if (std::filesystem::exists(DOSYA_OGRENCI)) {
    bool ok = dosyadan_yukle(DOSYA_OGRENCI, ogrenciDepo)
            && dosyadan_yukle(DOSYA_DERS, dersDepo)
            && dosyadan_yukle(DOSYA_KAYIT, dersKaydiDepo);
    if (ok)
        std::cout << std::format(
            "[BILGI] {} ogrenci, {} ders yuklendi.\n",
            ogrenciDepo.boyut(), dersDepo.boyut());
}

// ... islemler ...

// Dosyaya kaydet
bool kayit = dosyaya_kaydet(DOSYA_OGRENCI, ogrenciDepo)
           && dosyaya_kaydet(DOSYA_DERS, dersDepo)
           && dosyaya_kaydet(DOSYA_KAYIT, dersKaydiDepo);
if (kayit)
    std::cout << "[BILGI] Veriler kaydedildi.\n";
```

Akış

Başlangıçta **yükle** (yoksa hardcoded veri gir) → işlemleri yap → çıkışta **kaydet**

Derleme, Test ve Karşılaştırma

cmake && make Akışı

Derleme Adımları

1. `mkdir build && cd build` – derleme dizini oluştur
2. `cmake ..` – Makefile üret
3. `make` – derle
4. `./Uygulama3` – çalıştır
5. `./TestUygulama3` – testleri koş

```
$ mkdir build && cd build
$ cmake ..
-- The CXX compiler identification is GNU 13.2.0
-- Configuring done
-- Generating done
$ make
[100%] Built target Uygulama3
[100%] Built target TestUygulama3
$ ./TestUygulama3
=== OBYS Modern - Birim Testleri ===
  [OK] Tum testler basarili!
```

assert() ile Birim Test: AAA Kalıbı

```
void test_depo_ekle_bul() {
    // Arrange: Depoyu hazirla
    Depo<int, std::string> depo;

    // Act: Islem yap
    depo.ekle(1, "Ahmet");
    depo.ekle(2, "Ayse");

    // Assert: Sonucu dogrula
    assert(depo.boyut() == 2);
    assert(depo.iceriyor_mu(1));
    assert(!depo.iceriyor_mu(3));

    auto sonuc = depo.bul(1);
    assert(sonuc.has_value());
    assert(sonuc.value() == "Ahmet");

    auto yok = depo.bul(99);
    assert(!yok.has_value());

    std::cout << "  [OK] test_depo_ekle_bul\n";
}
```

Dosyalama Round-Trip Testi

```
void test_dosyalama() {
    const std::string gecici = "_test_depo.bin";

    // 1. Kaynak depo olustur
    Depo<int, Ogrenci> kaynak;
    kaynak.ekle(1001,
        {1001, "Ali", "Veli", "2000-01-01", "erkek"});

    // 2. Dosyaya kaydet
    assert(dosyaya_kaydet(gecici, kaynak));

    // 3. Baska depoya yukle
    Depo<int, Ogrenci> hedef;
    assert(dosyadan_yukle(gecici, hedef));

    // 4. Dogrula
    assert(hedef.boyut() == kaynak.boyut());
    assert(hedef.getir(1001).isim == "Ali");

    // 5. Gecici dosyayi sil
    std::remove(gecici.c_str());
    std::cout << "  [OK] test_dosyalama\n";
}
```

Round-Trip

Yaz → Oku → Karşılaştır. Veri bozulmadan dönüyorsa serileştirme doğru!

Eski / Yeni Yapı Karşılaştırması

Özellik	Eski (Bölüm 2)	Yeni (Ünite 5)
Varlıklar	`class` + getter/setter	`struct` (flat)
Veri yapısı	`vector<T>`	`Depo<K,V>` (map tabanlı)
Arama	$O(n)$ lineer	$O(\log n)$ map
Hata yönetimi	Hata kodu / `nullptr`	`optional` + istisna hiyerarşisi
Not sistemi	`double`	`variant<int, string, Muaf>`
String param.	`const string&`	`string_view`
Çıktı	`cout «`	`std::format`
Sorgulama	Manuel döngü	`ranges` pipeline
Dosyalama	Yok	`reinterpret_cast` binary I/O
Test	Yok	`assert()` ile AAA
Derleme	Elle g++	CMake

Evrim

Tek dosya, tek sınıf → modüler, test edilebilir, serileştirilebilir proje.

Özet

Bu Hafta Öğrendiklerimiz

Proje Yapısı:

1. CMake ile çok dosyalı proje
2. include/src ayrımı
3. İki hedef: program + test

Varlık & Depo:

4. Flat struct tasarımı
5. `Depo<K,V>` template
6. `bul()` / `getir()` stratejisi

Servis & Dosyalama:

7. Ranges pipeline sorguları
8. `string_view` + `format`
9. Binary I/O: POD + string + variant
10. `map` ve `Depo` serileştirme

Test:

11. `assert()` ile AAA kalıbı
12. Round-trip doğrulama

Gelecek Hafta: Qt Framework'üne Giriş

- Qt nedir? Çapraz platform GUI çerçevesi
- İlk Qt penceresi: QApplication ve QWidget
- Sinyal/Slot mekanizması: olay tabanlı programlama
- Widget'lar: buton, etiket, metin kutusu
- Düzen yönetimi: QHBoxLayout, QVBoxLayout
- OBYS: Konsoldan masaüstü uygulamasına geçiş!

Hazırlık

Kitabın Bölüm 4'ü okuyun. Qt 6 kurulumunu yapın. Bu haftaki OBYS projesini CMake ile derleyip çalıştırın.